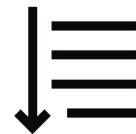


ES.next

Nieuwe JavaScript mogelijkheden

Agenda



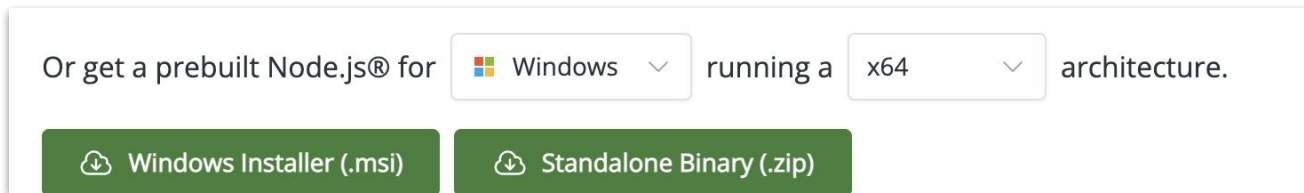
1. JavaScript releases
2. Waarden testen en toekennen
3. Arrays en functies
4. ...spread en ...rest operator

Installeer Node.js



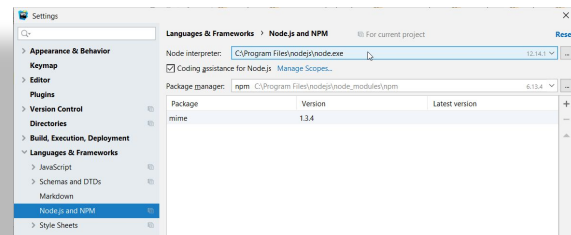
Download van nodejs.org en installeer de laatste LTS release

- een Node LTS release heeft altijd een PAAR(*even*) hoofdnummer



Webstorm vindt normaal je node.js installatie

- Settings (ALT+CTRL+S)
 - Languages & Frameworks > Node.js
- Je kan coding assistance aanzetten
- om Javascript rechtstreeks op je PC te draaien



JavaScript releases

JavaScript releases

- ES2015 (ES6) was een mega release waar JavaScript veel nieuwe functies kreeg die je in populaire OO talen vindt
- Sindsdien ES2016 wordt met jaarlijkse, kleinere releases gewerkt
 - Elk nieuw voorstel gaat door een pijplijn met verschillende stadia
 - <https://tc39.github.io/process-document/>
 - Een mooi overzicht van jaarlijkse nieuwe features sinds ES2016: https://exploringjs.com/js/book/ch_new-javascript-features.html

In welke browser werken nieuwe JS functies?

- Je doelpubliek heeft niet altijd de laatste browser versie
 - browsers werken tegenwoordig ook met frequente, automatische, kleine updates
- compat-table.github.io/compat-table

COMPAT ES

ECMAScript

5

6

2016+

next

intl

non-standard

compatibility table

Flattr

by kangax & webbedspace & zloirock

Fork

553

Sort by

Engine types

Show obsolete platforms

Show unstable platforms

V8

SpiderMonkey

JavaScriptCore

Chakra

Other

Minor difference (1 point)

Small feature (2 points)

Medium feature (4 points)

Large feature (8 points)

		Compilers/polyfills						Desktop browsers													
		92%	9%	45%	32%	37%	15%	3%	72%	87%	87%	75%	95%	92%	92%	92%	100%	100%	100%	100%	89%
Feature name	Current browser	Traceur	Babel+core-js ^[2]	Closure	Type-Script+core-js	es7-shim	IE 11	Edge 15	Edge 16	Edge 17 Preview	FF 52 ESR	FF 56	FF 57	FF 58 Beta	FF 59 Nightly	CH 62, OP 49 ^[1]	CH 63, OP 50 ^[1]	CH 64, OP 51 ^[1]	CH 65, OP 52 ^[1]	SF 10.1	
2016 features																					
• exponentiation (**) operator	3/3	2/3	3/3	3/3	2/3	0/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	
• Array.prototype.includes	3/3	0/3	3/3	1/3	3/3	2/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	

In welke release stage zit een ES.next feature?

COMPAT ES ECMAScript 5 6 2016+ next intl non-standard compatibility table Flattr by kangax &

This table shows proposals which have not yet been included in the current ECMAScript standard, but are at one of the maturity stages of the TC39 process.

Sort by Engine types Show obsolete platforms Show unstable platforms

V8

SpiderMonkey

JavaScriptCore

Chakra

Other

Minor difference (1 point)

Small feature (2 points)

Medium feature (4 points)

Feature name	Current browser	Compilers/polyfills					Desktop browsers											
		Traceur	Babel + core-js	Closure	Type-Script + core-js	IE 11	Edge 15	Edge 16	Edge 17 Preview	FF 52 ESR	FF 56	FF 57	FF 58 Beta	FF 59 Nightly	CH 62, OP 49	CH 63, OP 50		
Candidate (stage 3)																		
object rest/spread properties	2/2	0/2	2/2	0/2	2/2	0/2	0/2	0/2	0/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2		
global	0/2	0/2	2/2	0/2	2/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2		
Asynchronous iterators	2/2	0/2	2/2	0/2	2/2	0/2	0/2	0/2	0/2	0/2	2/2	2/2	2/2	0/2	2/2	2/2		
RegExp named capture groups	No	No	Yes	No	No	No	No	No	No	No	No	No	No	Flag	Flag	Flag		
RegExp Lookbehind Assertions	No	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes		
RegExp Unicode Property Escapes	No	No	Yes	No	No	No	No	No	No	No	No	No	No	Flag	Flag	Flag		
instance class fields	0/3	1/3	1/3	0/3	1/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3		
Function.prototype.toString revision	7/7	0/7	0/7	0/7	0/7	1/7	4/7	4/7	4/7	2/7	7/7	7/7	7/7	4/7	4/7	4/7		
Promise.prototype.finally	0/3	0/3	3/3	0/3	3/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	3/3	3/3	0/3	3/3		
Array.prototype.flatten, flatMap	0/2	0/2	2/2	0/2	2/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2	0/2		
optional catch binding	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	3/3	3/3	0/3	0/3		
numeric separators	No	No	Yes	No	No	No	No	No	No	No	No	No	No	No	No	No		
Draft (stage 2)																		

MDN browser compatibiliteit

- [MDN](#) geeft bovenaan elke pagina [baseline status](#) en onderaan ook de browser compatibiliteit van een feature

if()



Limited availability



Experimental: This is an [experimental technology](#).

Check the [Browser compatibility table](#) carefully before using this in production.

The `if()` [CSS function](#) allows different values to be set for a property depending on the result of a conditional test. The test can be based on a [style query](#), a [media query](#), or a [feature query](#).

Browser compatibility

[Report problems with this compatibility data](#) • [View data on GitHub](#)

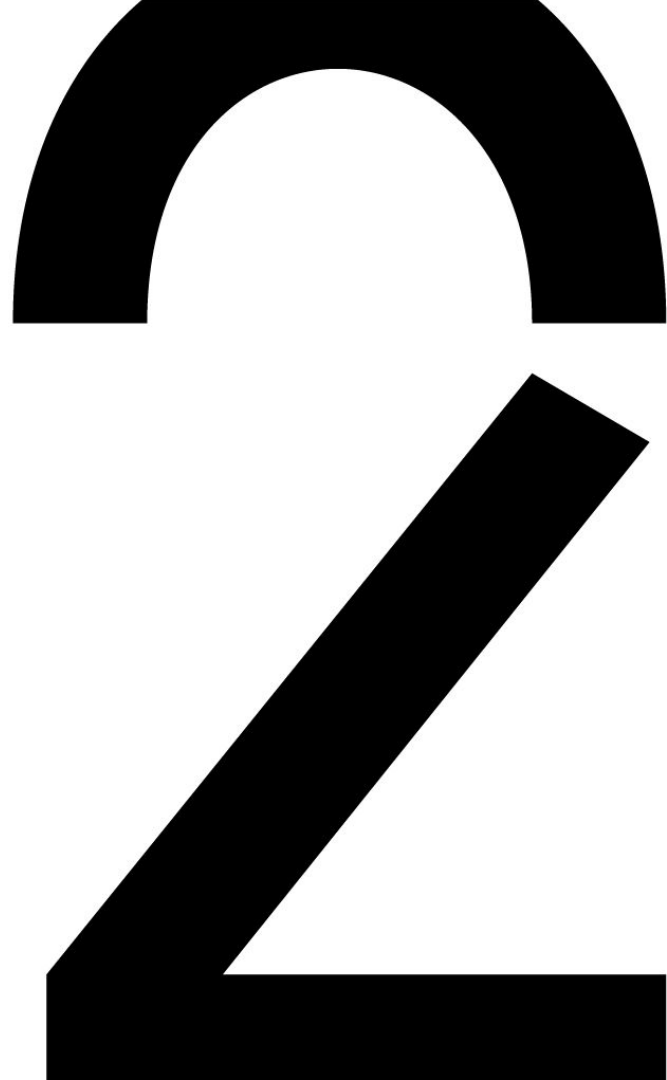
	Desktop					Mobile						
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet	WebView Android	WebView on iOS
if()	✓ 137	✓ 137	✗ No	✓ 121	✗ No *	✓ 137	✗ No	✓ 90	✗ No *	✗ No	✓ 137	✗ No *

Selectie

- We pikken er de interessantste features uit
- Asynchrone functies en modules worden in latere hoofdstukken behandeld



Waarden testen en toekennen



```
nullish.js
```

```
let bottle
console.log(bottle) // undefined
console.log(bottle?"bottle is empty") // bottle is empty
console.log(bottle.size) // Uncaught TypeError: bottle is
undefined. ABORTS!
console.log(bottle?.size) //undefined
console.log(bottle?.size?"1 liter") //1 liter
```



- ?? Als voorgaande waarde undefined of null is, wordt de waarde na de operator gebruikt



- in tegenstelling tot ternary operator (bottle?true:false)die true geeft bij undefined
- ?. object.attribuut op een undefined object geeft een exception (stopt het programma). Als je een vraagteken voor het punt plaatst, wordt voor het attribuut de waarde undefined teruggegeven (het programma stopt niet)

Null conditional assignment

ES 2021

```
nullish.js
```

```
let drink
drink ??= "water" // assign als drink null / undefined is
console.log(drink) // water
drink ??= "wine"
console.log(drink) // water
drink = "rum"
drink ||= "wine" // assign als drink falsy is
console.log(drink) // rum
```

- **&&=** ken een waarde toe als een variabele truthy is

Object literal: korte notatie

ES 2015

- ES2015 bevat een korte notatie om een object waarden te geven bij declaratie
- We bekijken eerst de klassieke en dan de korte notatie



Object literal: klassieke notatie

literal.js

```
let name = "Elisabeth";  
let age = 38;  
let herProfile = {  
  name: name,  
  age: age,  
  display: function() {  
    console.log(`${this.name} is ${this.age}`);  
  }  
}
```

Object literal: korte notatie

literal.js

```
let name = "Elisabeth";
```

```
let age = 38;
```

```
let herProfile = {
```

```
  name,
```

```
  age,
```

```
  display: function() {
```

```
    console.log(`${this.name} is ${this.age}`);
```

```
  }
```

```
}
```

Krijgen automatisch waarde van
variabele met dezelfde naam



Voorkennis

Object literal: Klassieke functie notatie

```
literal.js
```

```
let name = "Elisabeth";  
let age = 38;  
let herProfile = {  
  name: name,  
  age: age,  
  display: function() {  
    console.log(`${this.name} is ${this.age}`);  
  }  
}
```




Voorkennis

Object literal: Klassieke functie notatie

```
literal.js
```

```
let name = "Elisabeth";
```

```
let age = 38;
```

```
let herProfile = {
```

```
  name: name,
```

```
  age: age,
```

```
  display: function() {
```

```
    console.log(`${this.name} is ${this.age}`);
```

```
  }
```

```
}
```

Anonieme **function** toegekend aan object attribuut

Object literal: korte functie notatie

ES 2015

literal.js

```
let name = "Elisabeth";
```

```
let age = 38;
```

```
let herProfile = {
```

```
  name,
```

```
  age,
```

```
  display () {
```

```
    console.log(`${this.name} is ${this.age}`);
```

```
  }
```

```
}
```

~~function~~

- Een manier om meerdere attributen tegelijk uit een object op te halen
- We bekijken eerst de klassieke manier om dit te doen, en dan de nieuwe notatie

Klassieke toekenning attribuut aan variabele

```
let herProfile = {  
  name: "Elisabeth",  
  age: 96  
};  
let name=herProfile.name;  
let age=herProfile.age;
```

één instructie kent twee (of meer) waarden toe aan de attributen van een object

meerdere instructies nodig om waarden in attributen van een object aan twee (of meer) variabelen toe te kennen



ALT+ENTER zet om naar destructuring

```
let herProfile = {  
  name: "Elisabeth",  
  age: 96  
};  
let {age, name} = herProfile;
```

één instructie kent waarden in
object attributen aan meerdere
variabelen toe

- Werkt ook voor functies

```
let herProfile = {  
  name: "Elisabeth",  
  age: 96,  
  display() {...},  
};  
let {age, name, display} =  
  herProfile;
```

destructure.js

ES 2015

destructure.js

```
let herProfile = {  
  name: "Elisabeth",  
  age: 96  
};  
let {age} = herProfile;  
let {name:x,age:y} = herProfile;  
console.log(x,y); // Elisabeth 96  
let {age=18,familyName="de Grote",name="Karel"} = herProfile;  
console.log(age, name, familyName); // 96 Elisabeth de Grote
```

Is hetzelfde als

```
let x = name;
```

```
let y = age;
```

attribuutnaam altijd eerst in JS object ':'
notatie



destructureren

```
destructure.js
```

```
let herProfile = {  
  name: "Elisabeth",  
  age: 96 ,  
  address: {  
    zip: 2000,  
    community: "Antwerp"  
  }  
};  
  
let {address:{community}} = herProfile;  
console.log(community); //Antwerp
```

destructureren van een array

ES 2015

```
destructure.js
```

```
let tel = ["een", "twee", "drie", "vier", "vijf"];  
let [pink,,middenvinger] = tel;  
console.log(pink,middenvinger); //een, drie
```

Arrays en functies



Closures

- Javascript functies kunnen aan variabelen toegekend worden, aan andere functies doorgegeven worden als parameter en door andere functies teruggeven worden. Dergelijke functies noemt men **hogere orde functies**.
- Javascript functies zijn **closures**: ze hebben toegang tot de (variabelen in de) context waarin ze aangemaakt zijn (de **enclosing context**)

closuresCrows.js

```
let birds = "magpies";  
function showMessage(number) {  
    console.log(number + " " + birds);  
}
```

```
showMessage(10); //10 magpies  
birds="crows"  
showMessage(9); //9 crows
```

```
function makeCounter() {  
    let count = 2;  
    return function () {return count;  
}
```

```
let counter = makeCounter();  
showMessage(counter()); // 2  
showMessage(counter()); // 3  
let recounter = makeCounter();  
showMessage(recounter()); // 2
```

`showMessage` heeft toegang tot `birds` (buiten de functie gedefinieerd)

Ook als `birds` verandert

`makeCounter` retournt een anonieme functie

Deze functie retournt een nummer

`counter` bevat de functie die door `makeCounter` teruggegeven wordt

Ook na het einde van `makeCounter()` heeft `counter()` toegang tot de `count` variable

`recounter` verwijst naar een nieuwe `count` variable



Voorkennis

Itereren over een array: forEach

- Array functie parameter die op elk element van de array een actie uitvoert

itArray.js

```
let tabel = ["appel", "peer"];  
tabel.forEach(function(element) {  
    console.log(element);  
})  
// Het zelfde met een lambda  
tabel.forEach(element => console.log(element));
```

appel
peer

Lambda fat arrow syntax

Itereren over een array: map

ES 5

- Map itereert over de array en heeft als parameter een functie die elk element verandert.
 - Map geeft een array met alle nieuwe waarden terug.

itArray.js

```
let names = [ "Groucho", "Karl", "Ingeborg"];  
console.log(names.map(name ⇒ name + " Marx" ));  
// ["Groucho Marx", "Karl Marx", "Ingeborg Marx"]
```

Voeg aan elke naam een achternaam toe

Itereren over een array: filter

ES 5

- Filter heeft als parameter een functie die voor elk element `true` of `false` teruggeeft
- Filter verwijdert de elementen waarvoor de functie `false` gaf

```
itArray.js
```

```
console.log([ "Groucho", "Karl", "Ingeborg"]  
    .filter(value => value.includes("a"))  
);  
// ["Karl"]
```

Behoud alleen namen
die een "a" bevatten

Itereren over een array: reduce

ES 5

- Verwerk de hele array tot één resultaat
 - Eerste parameter (accumulator) houdt tussenresultaat bij
 - Het eerste element is de startwaarde voor de accumulator.

```
itArray.js
```

```
let nrs = [10, 20, 30, 40, 50];  
console.log(nrs.reduce ( (acc,val) => acc+val));  
// 150
```

Itereren over een array: reduce

- De startwaarde voor de accumulator kan als optionele tweede parameter aan reduce gegeven worden.
 - het eerste element van de array heeft nu geen speciale betekenis

```
itArray.js
```

```
let nrs = [10, 20, 30, 40, 50];  
nrs.reduce(  
  (acc, val) => acc + val, // eerste parameter is een functie  
  50                      // tweede parameter is startwaarde  
);  
// 200 (iteraties: 1. 50+10, 2: 60+20, 3: 80+30, 4: 110+40...)
```


- **find, findLast**: zoals filter, maar geeft één element terug met eerste/laatste match terug
- **findIndex, findLastIndex**: zoals find, maar geeft positie eerste/laatste match terug
- gespecialiseerde accumulator functies (ipv reduce): sum, join, groupBy...
- En vele andere functies...

Itereren over een array: ketting

ES 5

- Enkele array functies geven opnieuw een array als resultaat (map, filter, sort...). Daarop kan je een nieuwe array functie oproepen.
 - Zo vorm je een keten van functies (function chaining), wat een vloeiende programmeerstijl geeft.
- Vergelijkbaar met streams in java

Itereren over een array: ketting



Voorbeeld

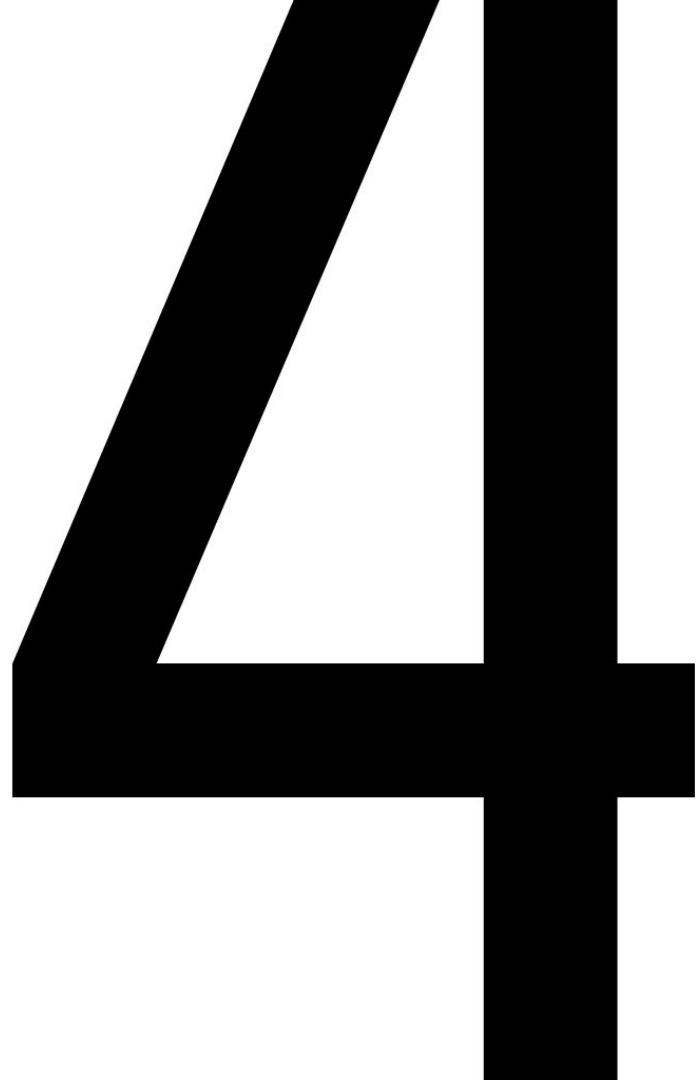
ES 5

```
arrayArrow.js
```

```
["Groucho", "Karl", "Ingeborg"]  
  .filter(value => !value.includes("a")) // ["Groucho",  
  "Ingeborg"]  
  .map(value => value.length) // [8, 7]  
  .sort( ) // function not needed: [7, 8]  
  .forEach(a => console.log(a));
```

- sort ordent op de waarde van de elementen (numeriek, alfabetisch...)
- Voor een andere sortering geef je aan sort een functie mee met twee parameters. Geef een positief getal terug als het eerste element groter is, een negatief als het kleiner is.

...Spread en ...Rest operator



Spread: ...array ⇨ elementen

- door iterable **...a toe te kennen** wordt ...a **uitgepakt** (*verspreid*) en worden de afzonderlijke waarden toegekend

```
spread.js
```

```
let a = [2,3,4];  
console.log([ 1, a, 5 ]);  
// [ 1, [ 2, 3, 4 ], 5 ]  
console.log([ 1, ...a, 5 ]);  
// [ 1, 2, 3, 4, 5 ]
```

```
function bar(y,z) {  
  console.log("spread", y, z );  
}  
  
bar( ...[1,2] );  
//spread 1 2
```

Rest ...array ⇨ elementen

- *resterende* parameters van een functie worden **aan ...z toegekend** en **ingepakt** in een iterable.

```
function foo(y, ...z) {  
  console.log("//rest ", y, z ); }  
  
foo( 1, 2, 3, 4, 5 );  
//rest 1 [2,3,4,5]
```

Spread: ...object ⇨ attributen

- Je kan met spread de attributen uit een object te halen

```
let herProfile = { name: "Elisabeth", age: 96 };
let herAddress = { name: "Liza",
  address: { zip: 2000, community: "Antwerp" }
};
let all = {...herProfile, ...herAddress}
// all: { name: 'Liza', age: 96,
//   address: { zip: 2000, community: 'Antwerp' } }
```

herProfile en **herAddress** worden gespread en de uitgepakte attributen gemerged in een nieuw object

Rest ...object ⇐ elementen

- je kan de rest operator gebruiken bij destructuren
- Handig om attributen (hier *name*) uit een object weg te filteren

restSpread.js

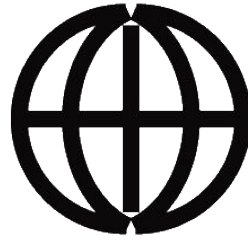
```
// all: {  
//   name: 'Liza',  
//   age: 96,  
//   address: { zip: 2000,  
//             community: 'Antwerp' }}  
let {name, ...anonymous} = all;  
  
console.log("name= ", name);  
console.log("anonymous= ",  
            anonymous);
```

```
// name = Liza  
// anonymous = { age: 96,  
//   address: { zip: 2000,  
//   community: 'Antwerp' } }
```




Overzicht

1. JavaScript releases
2. Waarden testen en toekennen
3. Arrays en functies
4. ...spread en ...rest operator



Online

- <https://compat-table.github.io/compat-table/>

 [Mozilla Developer Network / JavaScript](#)

 [Javascript.info tutorial](#)